

Fuel Pressure Sensor and Alarm Module High-Level Design (HLD)

Version: 1.0

Date: 16-03-2026

Prepared by: Karthika V T

1. Purpose

This document serves as a blueprint for the Fuel Pressure and Alarm system. The design document is created based on the system requirements. Based on the requirements (monitor fuel pressure, triggers alarms) the system design is implemented. The system is intended to continuously monitor fuel pressure levels in real time, detecting under pressure and over pressure criteria and notify the alarm.

2. System Overview

The system consists of a sensor module used for measuring the system pressure, a validation unit used to validate the range of pressure in the threshold range and an alarm system to notify whenever an under pressure or over pressure case occurs.

2.1 Key Components

2.1.1 Sensor Module

This module measures the pressure of the system and sends it to the validation unit. It is used to manage the raw communication between the physical fuel pressure sensor and the MCU. It ensures the incoming signal is captured accurately and consistently and converts raw voltage or binary values into meaningful units. The module contains a pressure sensor and a standard communication protocol is implemented.

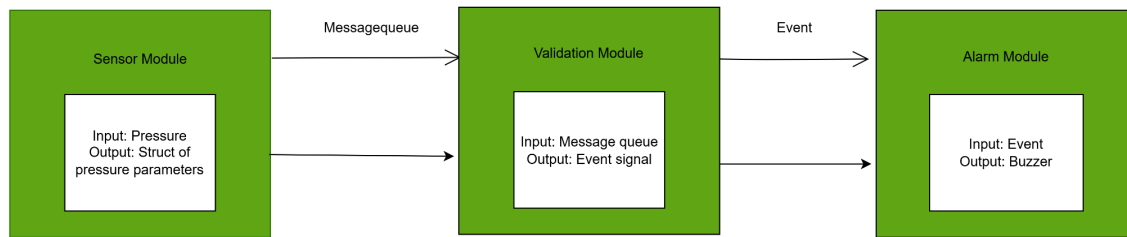
2.1.2 Validation Unit

This module is the decision making core of the system. Its primary function is to perform automated limit checking by comparing incoming data against hardcoded threshold values. It takes a unified pressure value from the message queue and compares with the predefined constants. If it is out of the range an event signal is created to trigger the alarm.

2.1.3 Alarm Module

This module is responsible for the fault indication. Whenever an event signal is given to the alarm module, it will trigger the buzzer and the user can reset the alarm. The module contains a buzzer with an DAC which converts the digital signal to the analog alarm sound. The reset register can be set using the reset button to stop the alarm.

2.2 High Level Architecture



3. Software Architecture

The system is designed in a layered architecture, such that operations are segregated into different modules and output of one module is given to the input of another.

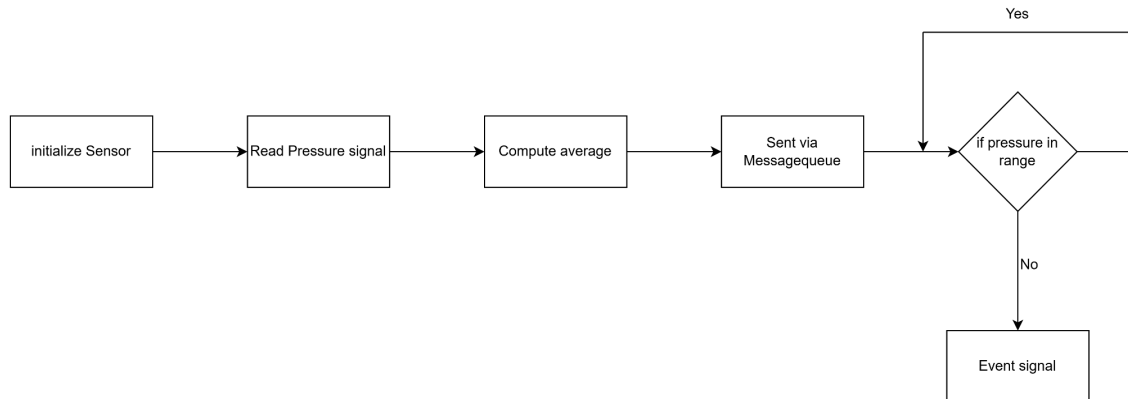
3.1 Major modules

Module	Functionality	Input/Output
Sensor Module	Inputs the pressure of the system and data is saved in the message queue and shared to the validation unit	Input: Pressure of the system Output: Data shared via message queue
Validation Module	Pressure value is compared with the MIN_PRESSURE and MAX_PRESSURE, if it is out of range event signal is triggered to the alarm module	Input: Pressure data from sensor module Output: Event signal to the alarm module
Alarm Module	Module for fault detection, event signal trigger the alarm and user can reset the alarm	Input: Event signal from validation unit Output: Alarm signal

3.2 Data Flow



3.3 Control Flow



4. Module Descriptions

4.1 Sensor Module

This module measures the pressure of the system and sends it to the validation unit. It consists of a pressure sensor of f frequency, n bit resolution and s sampling rate. It measures the analog pressure signal for a t time duration and the signal is sampled based on the s value. Thus discrete samples are obtained and the average of these samples is taken to get the unified pressure value. This pressure value is inserted into a message queue and shared to the validation unit.

4.1.1 Purpose

The purpose of this module is to quantify and digitize system pressure for downstream validation. It captures high frequency analog signals over a specific duration, converts them into discrete data points and calculates an average pressure to filter out noise. This processed value is then buffered into a message queue to ensure reliable communication with the validation unit.

4.1.2 Key Interfaces

The module contains an analog sensor interface, digital processing interface and a messaging interface. The analog sensor interface inputs continuous analog voltage or current from the pressure sensor. The digital processing interface contains the timing logic that governs the

duration for data acquisition. The messaging interface is the primary software interface used to hand off unified value to the validation unit.

4.2 Validation Unit

The validation unit contains the main logic for the system management. The threshold values MIN_PRESSURE and MAX_PRESSURE are predefined in the code. The pressure value input from the message queue is compared with the threshold values. If the value is within the threshold values the operation continues as it is. Whenever the pressure value comes below MIN_PRESSURE or goes beyond MAX_PRESSURE an event will be generated and the signal is given to the alarm module.

4.2.1 Purpose

The validation unit is responsible for real time safety monitoring and operational control. By comparing processed data from the message queue against predefined values, it ensures the system remains within a safe physical limit. Its core purpose is to provide automated anomaly detection, maintaining steady state operations during nominal conditions while immediately triggering the alarm module whenever a threshold violation occurs.

4.2.2 Key Interfaces

The module consists of input interface, logic interface and an output interface. The input interface is the primary software interface that listens to the queue and retrieves the unified pressure value sent by the sensor module. The data is shared in a structured object or a packet. The logic interface is where threshold values are hardcoded to define the permissible limit. The output interface is a hardware trigger or a GPIO signal sent to the alarm module.

4.3 Alarm Module

The alarm module serves as the system's critical safety interface, providing immediate auditory feedback when a pressure violation occurs. Its primary function is to translate digital error signals from the validation unit into a physical alert using an integrated DAC. To ensure acknowledgement, the module maintains a latched alarm state that persists until a manual intervention occurs. By monitoring a physical reset button, the module updates an internal reset register, allowing users to silence the alarm and acknowledge the fault once the system is under control.

4.3.1 Purpose

The purpose of the alarm module is to provide real time fault notification and user acknowledgement for the system. It acts as the final safety layer by converting digital event triggers into an audible analog signal via DAC driven buzzer, ensuring that pressure anomalies are immediately perceptible to the user.

4.2.2 Key Interfaces

The alarm module consists of the input interface, analog audio interface and user interaction interface. The internal DAC connection that converts software sound patterns into a varying voltage for the buzzer. The physical connection to the buzzer hardware, delivering the analog

signal to produce the alarm sound. The user interaction interface is a physical or digital input interface used to detect when the user has pressed the button. A specific memory mapped location or a variable that stores the acknowledged state, allowing the software to stop the alarm.

5. Interface Design

5.1 Application Programming Interface

5.1.1 Sensor APIs

`bool initPressure()` - configure the sensor module and enable it.

`uint32_t readPressure()` - calculate the pressure value from the input signal

5.1.2 Alarm control APIs

`bool initBuzzer()` - configure the buzzer and start the operation

`bool triggerBuzzer(uint32_t event)` - output registers of the buzzer are enabled , resulting in triggering an alarm.

`bool resetAlarm()` - Alarm can be reset once the user is acknowledged.

`bool returnStatus()` - Alarm status can be returned for verification.

5.1.3 Display APIs

`bool initDisplay()` - the display is configured and enabled

`uint32_t updateDisplay(value, status, length)` - the data sent is displayed on the LCD

`bool clearDisplay()` - used to clear the LCD

5.2 Inter-Process Communication Methods

5.2.1 Message Queue

This is the primary link between the sensor module and validation unit. It acts as a buffer for the pressure value. It decouples the high frequency sampling logic of the sensor from the decision making logic of the validation unit. This ensures that even if the validation unit is busy, sensor data is not lost, it is queued for processing.

5.2.2 Event Signaling

This mechanism connects the validation unit to the alarm module. When a threshold is breached, an event signal is generated. The event signal is a lightweight notification meant for immediate action. It forces the alarm module to respond instantly to a fault condition, bypassing standard data processing delays.

5.3 Data Exchange Formats

The data undergoes two major transformations as it moves from the physical world to the user interface. Before the pressure value is unified, it exists as a bit stream defined by the n bit resolution. These are raw, discrete samples captured at the s sampling rate over t duration. This is the primary data exchange format between the sensor module and the validation unit. It will be a C struct or JSON object containing a floating point number representing the average pressure value. A timestamp or sensor ID to ensure data traceability. This message is pushed into the message queue, allowing the validation unit to consume it asynchronously.

6. Data Flow



7. Error Handling & Fault Management

7.1 Fault detection

Sensor validation - Detect missing, stuck, or out of range pressure values

Signal integrity check - Identifies noise, spikes, or unstable readings

Communication monitoring - Detects timeouts or corrupted packets

7.2 Fault isolation

Module level isolation - If sensor fails, system disables its input and switches to fallback.

Error tagging - Faults are labeled with source identifiers (SENSOR_FAULT, COMM_FAULT)

Event queue filtering - Only relevant fault events are passed to the alarm module

7.2 Fault recovery

Reattempt sensor reads or communication after short delay

Manual reset - User presses physical button to acknowledge and clear alarm

Status update - Reset register updated to reflect recovery state

8. Traceability

This document has been prepared based on the SRS to ensure that all design elements, data flows, control logic, and fault management mechanisms trace back to the defined requirements.